Toward Fewer Than 196 States: Comparison, Exact Rewrites, and Verification Boundaries

Comparative research and executable certificates

13 November 2026

Abstract

The strict target of this investigation is a binary, single-tape machine with fewer than 196 working states whose blank-tape nonhalting is equivalent to the consistency of ordinary first-order Peano arithmetic. No such machine is established here. We compare the literal 208-, 209-, and 196-state constructions, provide two new explicit 196-state reader layouts with complete startup and executable symbolic checks, and give small certificates excluding specified repairs of five earlier near misses. A new equivalent axiom emitter uses 199 macro tokens and 3,773 primitive instructions instead of 201 and 3,812, with a uniform proof-translation argument verifiable in PA. An explicit 67-rule grammar is supplied; its shortest verified tape implementation here has 198 states and does not meet the target. A new source search permits jump depths to change while preserving the exact ordinal control graph. We distinguish these results from a smaller-machine construction and from an exported PA proof of the full consistency biconditional.

1 Target, conventions, and status

We use the same model as PA196: a deterministic Turing machine on a two-way infinite tape over $\{0, 1\}$, initially all zero, with one designated initial working state and one separate halting control. Each transition writes a bit, moves exactly one cell left or right, and changes control. The halt control is excluded from the working-state count. We do not change the alphabet, preload a nonblank program, or count only states seen in a finite simulation.

For a literal machine M, let H_M be its arithmetical blank-input halting sentence and $N_M = \neg H_M$. The strongest requested endpoint is

$$\mathrm{PA} \vdash N_M \leftrightarrow \mathrm{Con}(\mathrm{PA}), \qquad |Q_M| < 196.$$

A finite nonhalting prefix proves neither half of this statement. A state-count score for a source grammar is also insufficient: the physical interpreter, startup, and complete tape endpoints must still work.

The archive includes a fresh literal comparison, two explicit reader layouts in *rotated196*, an equivalent numerical axiom emitter and its grammar witness, and finite obstruction certificates for extending fixed failed folds. The rotation preserves 196 working states. Its symbolic checks use the unchanged register-engine contracts of the upstream development; they have not been exported as a new Lean build or an internal PA derivation. This qualification applies wherever the changed machine is discussed.

2 Pinned inputs and a uniform comparison

The inputs are frozen at the following revisions, so that future changes to `main` do not change this report:

- ProveIt: `f1f076e31995f10460838d1846456f4bb9c8ae80`.
- James Chang's PA196: `407cdc356f6b586635e71ac92be394255f4bfea4`.
- The requested PA209 predecessor: `69d98241a721ead20b4f0b17df9062950b40fbfe`.

The archive's `comparison/inputs/manifest.json` identifies each file, archive member, URL, and SHA-256 digest. The reference tables and relevant source programs are included with their notices. The independently written `comparison/compare_designs.py` checks the following results.

2.1 Nine U208 outputs represent two machines up to renaming

The five original construction packages designate nine final 208-state tables. They fall into exactly two rooted, action-preserving isomorphism classes, called A and B. The names describe classes only; no ordering of mathematical strength is intended.

Package	Principal table	Alternative table
`U208_ConPA`	A	—
`U208_ConPA_Conditional_Context`	A	B
`U208_ConPA_Entry_Context`	A	B
`U208_ConPA_History`	A	B
`u208_research`	B	A

All these packages are dated 29 October 2026. The checker constructs and verifies 16 complete state bijections: ten among the five A tables and six among the four B tables. Each bijection preserves the start and halt, read/write symbols, and movement directions. Byte differences between renumbered tables therefore do not identify independent savings that can be added together.

2.2 The broader incoming collection has twenty U208 classes

The two-class statement above does not describe the entire incoming directory. The later `pa208_research_2026-10-29.zip` supplies eighteen additional accepted 208-state helper machines: nine based on A and nine on B. All eighteen are pairwise nonisomorphic and distinct from A and B. Thus the complete accepted U208 collection audited here consists of **27 presentations in 20 rooted graph-isomorphism classes**.

These variants retain the respective A or B source program and calculus. Their extra helper distinguishes the bit left by an incoming transition, allowing two readers to share a state while paying for a new helper. The net state count remains 208. The new comparison checker independently checks each literal patch, every incoming-edge tag, the native-to-dense state map, and all required positive local blocks of one, three, or five steps. Each block touches only the current cell and its predecessor; it ends with the same complete tape and head as the corresponding parent step. The remaining rows commute with the state projection. This gives the stated bounded local halting equivalence, beyond graph comparison. All eighteen complete tables and their individual certificates are included.

The seven other incoming archives were also checked for additional accepted outputs. Besides these eighteen U208 machines, `pa_tm_research_2026-11-13.zip` supplies an existing alternative

196-state layout. The other five contain reference copies, rejected experiments, or logical investigations without an additional accepted literal machine. Their pinned status documents are included under `comparison/incoming_status/`. Including Chang's PA209 and PA196, the comparison checker reads **30 literal tables** and five distinct source layouts. The two new 196-state variants constructed in this report have separate verifiers.

2.3 Source metrics must use the same convention

Source	States	Sites	Internal	Leaf types	Total DAG	Max path
U208 A	208	3909	174	23	197	42
U208 B	208	3909	174	23	197	41
Chang PA209	209	3948	173	23	196	33
Chang PA196	196	3812	159	23	182	38
Incoming alternative196	196	3812	159	23	182	36

Here a *site* is one primitive instruction occurrence. Internal nodes are distinct binary-tree pairs; leaf types are distinct primitive instructions, including their register or jump-depth arguments. Total DAG size counts both. In particular, comparing the U208 value 197 directly with the PA196 value 159 mixes two conventions.

The two U208 sources have identical ordered primitive words and identical ordinal control graphs, while 720 binary instruction addresses differ. Chang's PA209 is a different source program from the U209 parent bundled with the U208 reports. A common ancestral tape engine does not make their literal state labels interchangeable.

The checker also independently replays the complete finite startup of Chang's machines. PA209 reaches its clean frame after 58,210 steps, at its literal root state 81; PA196 does so after 58,154 steps, at root state 75. Both heads are at absolute position -2, and their complete tapes agree relative to the head. The frame stores candidate 2, gap 48, and 26 spare blocks of value 1.

3 What the 196-state construction already saves

The published PA196 construction changes the axiom command and its layout, while retaining the 36-state tape engine of PA209. Its physical state budget is

$$\underbrace{36}_{\text{engine}} + \underbrace{13}_{\text{leaf routers}} + \underbrace{147}_{\text{readers}} = 196.$$

Its reader states implement 159 distinct internal source nodes. The existing context-dependent sharing is already counted in 147; its six axiom-body folds cannot be subtracted a second time. See the upstream paper for the logical recovery of the removed axiom head [1].

Let $p(a,b) = \binom{a+b+1}{2} + b$. PA196 uses

$$I(a,b) = 4p(a,b), \quad M(a,b) = 4p(a,b)+1, \quad U(a,B) = 4p(B,a)+2, \quad E(a,b) = 4p(a,b)+3.$$

Relative to PA209, the universal constructor transposes the pairing arguments. The axiom command drops the old seventh head, adds guarded equality vacuity, rearranges the other heads, and changes some antecedent orders. Its body has 201 macro tokens, represented by a 68-node binary grammar and 62 physical reader states after sharing. The complete source shrinks from 3,948 to 3,812 primitive occurrences.

U208's last one-state saving uses a different local pattern. In its native U209 parent, one predecessor enters the state to be deleted from opposite head directions, and a helper uses that

entry history to select the proper continuation. A direct scan of all PA196 rows finds no state whose two rows write 0 and move left, respectively write 1 and move right, into the same working target. Thus the exact mixed-entry U208 pattern cannot be transplanted by renaming. This finite fact does not rule out a more general history-dependent fold.

4 The consistency theorem: the exported direction matters

The public theorem inventory for PA196 explicitly includes

$$\mathrm{PA} \vdash N_{196} \to \mathrm{Con}(\mathrm{PA}). \tag{1}$$

Classically, this is equivalent to a refutation of PA yielding a halting trace of the literal machine. The published development also establishes external nonhalting and independence, with the stated metatheoretic consistency and soundness assumptions [2, 1]. The analogous PA209 inventory has the same internal direction.

The checked inventory does not supply the other internal implication

$$\mathrm{PA} \vdash \mathrm{Con}(\mathrm{PA}) \to N_{196}, \quad \text{equivalently} \quad \mathrm{PA} \vdash H_{196} \to \neg\,\mathrm{Con}(\mathrm{PA}). \tag{2}$$

This is an audit of the exported statements, not a claim that the converse is false or unprovable. ProveIt's existing `PA-BRIDGE.md` and November 13 manuscript already discuss this distinction [5]. It is confirmed here rather than presented as a new discovery.

An external semantic argument about each ordinary finite formula cannot simply be read as a single internal theorem for arbitrary coded formulas and traces in every model of PA. A satisfactory internal converse can instead use syntax: extract a finite command derivation from an arbitrary coded literal halting trace, translate its formulas and axiom instances into PA proofs, and verify the transformation uniformly. PA induction can then reason about the codes without a truth predicate for all formulas. The totality and correctness of the extraction and proof transformations are proof obligations, not consequences of a long finite run.

For a descendant machine, a PA-formalizable halting-preserving simulation would transport whatever endpoint is proved for its parent. It does not create a missing parent theorem. The present artifact neither exports a new internal biconditional nor treats a Lean metatheorem as a PA derivation.

5 A new explicit 196-state reader rotation

The successful concrete change acts outside the axiom body. In the upstream source notation, write $A = \texttt{node75}$, $B = \texttt{node37}$, and $C = \texttt{node163}$. Replace

$$((A, B), C) \quad \text{by} \quad (A, (B, C)).$$

The full ordinal control graph, including both branches of each decrement, remains identical. This is checked for all 3,812 sites; an association change alone would not justify that conclusion for a program with structured jumps.

5.1 The exact four-row change

All rows other than the following remain those of the pinned PA196 table. These are the *dense literal* labels, with start 0, root 75, and halt 196. Each entry is (write, move, next).

State	Read	Original	Rotated
88	0	$(0, R, 117)$	$(0, R, 126)$
88	1	$(1, R, 115)$	$(1, R, 117)$
117	0	$(0, R, 126)$	$(0, R, 28)$
117	1	$(1, R, 28)$	$(1, R, 115)$

For comparison with the native upstream reader names, the replacement rows are $(100, 0, 0, R, 213)$, $(100, 1, 1, R, 174)$, $(174, 0, 0, R, 28)$, and $(174, 1, 1, R, 164)$. Native start, root, and halt are 74, 84, and 254. The label conventions must not be mixed.

The complete 392-row dense table is `rotated196/candidate/data/rotated196.tm`. Its SHA-256 digest is

`25d686ab95f58b1d9eb1f75cb0960ca9f16640a1536140961c90ba8e103e8413`.

The native table, full source-site list, modified source definition, reference inputs, and verifier are included in the same directory. All 196 working states remain graph reachable; this is a same-size construction.

5.2 What was checked

The clean source seed is site 25 with logical register vector $(2, 0, 0, 0, 0, 0, 0, 0, 0)$. The new blank startup takes 58,294 steps, exactly 140 more than the original. The resulting state, head position, and entire finite-support tape equal the old clean configuration. Its physical file is $(0, 0, 0, 0, 0, 0, 2, 0, 0)$ followed by 26 ones, and its right marker is 48 cells to the right of the root head.

The executable symbolic verification checks 4,959 instruction/branch contracts. It performs 1,619,481 symbolic transitions, with a maximum of 515 per contract. These are *symbolic*, not literal tape-step counts. The checker retains arbitrary natural gap lengths and complete symbolic register files. It compares the complete intended output frame, including all tape contents and the head, rather than merely checking a return to the root control. All register-engine rows used by its inherited generic box contracts remain unchanged.

This gives a much stronger validation than a finite blank prefix. The ordinary composition argument is as follows. The clean configurations represent the same source seed. At each source site, both physical layouts execute a positive-length block to the source successor's full frame. The retained source control graphs agree exactly. By induction, finite source executions are represented in both layouts, with the same halt instruction. Positive block lengths and total primitive blocks are needed to turn this into both directions of halting preservation; they must not be replaced by endpoint guesses.

The certificate computation uses the generic register contracts transcribed from upstream `M6/Native.lean` and `M6/Runtime.lean`. It is an executable symbolic induction checker, not a new proof-assistant kernel check. Neither an end-to-end Lean proof of this literal changed table nor a PA consistency biconditional is claimed. The generated source definition is supplied for subsequent formal integration.

5.3 Why this rotation was worth constructing

The interpreter's return scan distinguishes its left marker using seven zeros. A proposed six-zero version removes one scanner state. In literal labels this would redirect row 23/0 from $(0, L, 34)$ to $(0, L, 69)$; in native labels the proposed new target is 75. It fails on the unmodified address layout, so it is *not* applied to the delivered machine.

The relevant source-address obstruction is the pattern `1000000`: a run of six zeros following a path one can imitate a marker. Six leading zeros can end at the actual left marker; seven leading zeros require separate exclusion. Checking only the maximum address length misses this issue. Address constraints are a search guide, not a substitute for the full primitive runtime checks.

Before the rotation, source site 3693 has address `1010101000000`. Afterwards it has `101010100000`. This removes the sole offending path outside the axiom body. Eighteen offending axiom-body addresses remain. The delivered 196-state table therefore keeps the original seven-zero scanner.

5.4 A second verified layout with adjusted jump depths

A second table combines the four-row rotation with a reassociation of the four-increment macro. Replace

$$((\texttt{node60}, \texttt{node60}), \texttt{continueTo}\ 2) \quad \text{by} \quad (\texttt{node60}, (\texttt{node60}, \texttt{continueTo}\ 3)).$$

The 64 changed continuation occurrences still have exactly their old ordinal destinations. Three additional native rows change:

$$(139, 0, 0, R, 99), \quad (139, 1, 1, R, 190), \quad (190, 1, 0, L, 42).$$

In dense literal labels these are $(111, 0, 0, R, 87)$, $(111, 1, 1, R, 121)$, and $(121, 1, 0, L, 41)$. The table has 196 working states and 392 rows. Its full source control graph is unchanged, all 4,959 symbolic frame contracts pass, and it reaches the same entire startup frame at step 58,294. The symbolic total is 1,618,329 steps, with maximum 515 per contract. Its dense-table SHA-256 is

`591f4240775256349f85a3d99708ba8ebd7e245779271c1cb2244cf04ce87e76`.

The directory `rotated196/candidate-macro62/` and `rotated196/verify_macro62.py` reproduce this second layout. It too retains eighteen marker obstructions. A subsequent exploratory search of 25-cell, 20-step local replacements found 54 candidate pairs from 167,651 symbolic contexts; none passed the complete instruction contracts. This bounded negative result is not an impossibility theorem for this layout.

6 New jump-retargeting source search

The prior directional-rotation search preserves structured jump depths. We enlarge the local search by permitting a jump's depth argument to change, while retaining its exact ordinal destination. For each reassociated tree:

1. retain the primitive instruction order and check every decrement's zero and positive successors;
2. for each forward or backward jump, enumerate the appropriate ancestor boundaries and choose the depth that reaches its old destination;
3. reject a missing destination or a required depth above 3;
4. rebuild the tree and compare its complete resolved ordinal graph with the original graph.

The source transformation is therefore a preservation of the resolved register program, not an assumption that tree associativity always preserves control flow.

There are 246 first global binary rotations of the distinct internal subtrees. Exactly 173 survive the full control-flow check. Of these, 12 require nontrivial jump retargeting beyond the prior criterion. Their internal-node/maximum-zero-run census is:

Internal nodes	Maximum zero run	Accepted first rotations
159	6	36
159	7	32
160	6	85
160	7	19
163	6	1

The 73 rejections comprise 46 changed decrement branches, 23 unrepresentable jump destinations, and four required depths above 3. None of the accepted first rotations has fewer than 159 internal nodes.

A second screen starts from the 11 newly retargeted first variants with at most 160 internal nodes and tests every second global rotation, applying the same repair and full graph comparison. It checks 2,706 ordered cases and finds no source below 159 internal nodes. This is not an exhaustive search of all second rotations from all first variants, and it says nothing about arbitrary machine layouts. Some retargeted variants introduce a jump instruction absent from the old physical routers; a fixed internal node count would still not guarantee a fixed machine-state count.

Reproduction is self-contained in `source_rotations/retarget_rotations.py`. The optional `-second` flag reproduces the second screen. The receipts omit machine-dependent elapsed times. Restricted physical allocation trials of selected variants produced no accepted smaller table; timeouts were not treated as unsatisfiability proofs.

7 Two-trace certificates for fixed near misses

Five earlier one-row near misses were used as fixed starting points for a new repair screen. The question is deliberately narrow: after making the specified state identification and keeping its first patched row fixed, can changing one additional transition repair every required local block?

The contexts come from the radius-six blank-start window overapproximation, which contains 270,067 state/windows. A block must match one projected parent step in at most 128 positive child steps, remain within the 13 cells $[-6, 6]$, and restore the complete represented tape and the required head position. The closure admits abstract halt, so it is not a nonhalting proof.

Drop	Keep	Fixed row replacement	Failing windows	Witness masks
45	160	$184/1 \mapsto (1, L, 75)$	416	99, 725
56	96	$96/1 \mapsto (1, R, 67)$	96	203, 3915
96	56	$56/1 \mapsto (1, R, 67)$	96	203, 3915
96	103	$103/1 \mapsto (1, R, 110)$	96	203, 331
103	96	$96/1 \mapsto (1, R, 110)$	96	203, 331

These are the original PA196 literal labels. Mask bit $i + 6$ is the bit at tape offset i.

Lemma 7.1 (Necessary intersection for a one-row repair)**.** *Fix a failed block and the exact required initial and final configurations. Any change to one transition address that alters the block before its specified failure must change an address queried by that block. Consequently, a single additional row capable of repairing several failed blocks must belong to the intersection of their queried-address sets.*

Proof. If the changed address is not queried, induction on executed steps gives exactly the same control, head, and tape at each step up to failure. The modified execution therefore has the same failure. Apply this argument to each required block. □

For each row of the table, two failed blocks have address-set intersection contained entirely in the already fixed patch. No eligible additional row can therefore repair both. The small JSON certificates retain the complete traces, parent endpoints, and window membership. A dictionary-based trace generator is checked by a separately implemented bit-vector replay. `repair_obstructions/repair_obstruction.py` recomputes the closure and verifies all five certificates.

The conclusion concerns these fixed first patches and this certificate template. It is not a lower bound on 195-state machines, an exhaustive unrestricted two-row search, or proof of different eventual halting behavior. Sharper reachable invariants, a wider matching region, multiple parent steps, a different tape representation, or two more changed rows remain outside it.

7.1 Two folds sharing a new helper

The eighteen additional U208 machines suggest a different way to spend a helper state: remove two old states and introduce one shared helper, so that $196-2+1=195$. The delivered `shared_helper/` component examines this mechanism directly on PA196. There are fourteen eligible pairs of states with opposite one-step predecessor tags and one common transition row, giving 88 disjoint pair choices. Both retained-state orientations are considered independently. Each helper target ranges over all 196 old working controls and the helper itself.

With bit-preserving, right-moving helper rows, the complete count is

$$88 \cdot 4 \cdot 197^2 = 13,660,768$$

combined patches. Allowing both write bits and both movement directions independently in each helper row gives sixteen action patterns and **218,572,288 patches**. No patch survives the exact whole-tape, two-cell, positive-block test with a 64-step bound. The broader family includes the narrower one; these counts should not be added as disjoint cases. The C++20 source and pinned-input Python replay wrapper are included. The search does not require either individual fold to succeed in isolation. It therefore tests actual interaction through the shared helper, including helper self-visits. Longer blocks, larger footprints, different entry predicates, or more changed rows remain outside this bounded result.

8 An equivalent axiom emitter with 39 fewer source instructions

A new source-level simplification distributes the innermost universal quantifier over the two conjuncts of PA196's separation condition. The result has **199 AX macro tokens instead of 201**, and expands to **3,773 source instructions instead of 3,812**. These are source counts. They establish no reduction in physical Turing-machine states.

Use the exact PA196 constructors

$$\mathsf{I}(x,y) = 4p(x,y), \quad \mathsf{M}(x,y) = 4p(x,y)+1, \quad \mathsf{U}(x,B) = 4p(B,x)+2, \quad \mathsf{E}(x,y) = 4p(x,y)+3,$$

where p is the upstream Cantor pairing, and put $\mathsf{N}(B) = \mathsf{I}(B,1)$ and $B \wedge C = \mathsf{N}(\mathsf{I}(B,\mathsf{N}(C)))$. The property parameter c is simultaneously a formula code and the fresh-output variable label, exactly as in the upstream emitter.

For the original gate parameters (a,b,c) define

$$\Delta = \mathsf{E}(a,b) \to \mathsf{U}(c,\mathsf{E}(a,b)), \qquad V = \mathsf{M}(a,b) \to \mathsf{U}(c,\mathsf{M}(a,b)),$$
$$P = c \to \mathsf{M}(c,b), \qquad A = \mathsf{M}(a,b) \to P, \qquad B = \mathsf{M}(c,b) \to c.$$

Here an arrow denotes I, and c in formula position denotes the property with numeric code c. The original definitions are

$$S = \mathsf{U}(b, A \wedge B), \qquad Q = \mathsf{U}(c, S \to \mathsf{N}(P)), \qquad F = \mathsf{N}\big(\Delta \to (V \to Q)\big).$$

Replace them by

$$S_1 = \mathsf{U}(b, A), \qquad S_2 = \mathsf{U}(b, B),$$
$$Q_* = \mathsf{U}\big(c, S_1 \to (S_2 \to \mathsf{N}(P))\big), \qquad F_* = \mathsf{N}\big(\Delta \to (V \to Q_*)\big).$$

The guard remains $c \neq a, b$; its disabled result remains truth code zero. The other six heads, the restricted propositional head, and the saved incoming/generalization entries are unchanged. In particular, the AX command still pushes nine items, and the head-selection ranks and fixed push/pop words require no change merely because of this logical reformulation. The numerical value of the enabled gate is different; the claim is derivable logical equivalence, not equality of codes.

Proposition 8.1 (Gate equivalence in the common numeric calculus)**.** *The retained heads h1, h6, the restricted implicational basis, modus ponens, and generalization derive both $F \to F_*$ and $F_* \to F$ for all numeric parameters. The proof does not use either version of the guarded gate, equality symmetry, a substitution rule, or unrestricted universal instantiation.*

Proof. The unchanged restricted implicational basis yields the classical implicational rules. With unchanged h1 it also yields explosion for code 1, hence the usual rules for the displayed negative encoding of conjunction. These are the upstream bootstrap arguments, independent of the gate.

For arbitrary formula codes X, Y and label b, generalize the propositional projections $(X \wedge Y) \to X$ and $(X \wedge Y) \to Y$. Applying h6 gives

$$\mathsf{U}(b, X \wedge Y) \to \mathsf{U}(b, X), \qquad \mathsf{U}(b, X \wedge Y) \to \mathsf{U}(b, Y).$$

Conversely, generalize the propositional theorem $X \to (Y \to X \wedge Y)$ and apply h6 twice. This gives

$$\mathsf{U}(b, X) \to \big(\mathsf{U}(b, Y) \to \mathsf{U}(b, X \wedge Y)\big).$$

Propositional reasoning therefore proves

$$\mathsf{U}(b, X \wedge Y) \;\leftrightarrow\; \big(\mathsf{U}(b, X) \wedge \mathsf{U}(b, Y)\big).$$

This is normal-quantifier distribution over a binary conjunction. It requires no freshness side condition.

Apply it with $X = A$ and $Y = B$. Currying the two conjuncts now gives

$$S \to \mathsf{N}(P) \quad \leftrightarrow \quad S_1 \to (S_2 \to \mathsf{N}(P)).$$

Generalize each direction at c and use h6 to obtain $Q \leftrightarrow Q_*$. Implicational reasoning transports this equivalence through the unchanged contexts $\mathsf{N}(\Delta \to (V \to \cdot))$, proving the result. □

The preceding proof is about the actual numerical constructors, not about a change of variable names in a decoded first-order language. The inner label b binds exactly the same occurrences on both sides of the conjunction-distribution law. If $a = b$, the resulting additional bound occurrences are present on both sides. The outer label c still binds the whole curried condition, and the occurrences of Δ and V remain outside it. The simultaneous use of c as a property code and a bound label is preserved. Thus no freshness assumption beyond the original gate guard enters, and no general substitution theorem is being used implicitly.

8.1 Uniform proof translation and its PA verification

Let C be the exact numerical proof calculus with the original gate, and let C_* replace only its gate by F_*. A proof is a finite list of labelled lines. An axiom line records its head and numerical parameters; an MP line records the two earlier line indices and its conclusion; a generalization line records an earlier line and its label. The checker verifies numerical equalities of the displayed constructors, the restriction $u \le a$ on the propositional head, and the original two inequalities on an enabled gate. One may code lists, for example, by $\mathrm{nil} = 0$ and $\mathrm{cons}(h,t) = 1 + p(h,t)$. Use any fixed tagged coding of these line records. Write $\mathrm{Prf}_C(p,f)$ and $\mathrm{Prf}_{C_*}(p,f)$ for the resulting primitive recursive proof predicates, with last conclusion f.

Theorem 8.2 (PA verification of the numerical proof translation)**.** *There are primitive recursive proof translators τ and σ for which PA proves*

$$\mathrm{Prf}_C(p,f) \longrightarrow \mathrm{Prf}_{C_*}(\tau(p),f),$$
$$\mathrm{Prf}_{C_*}(p,f) \longrightarrow \mathrm{Prf}_C(\sigma(p),f).$$

Consequently PA proves equality of the two refutability predicates,

$$\exists p\ \mathrm{Prf}_C(p,1) \quad \longleftrightarrow \quad \exists p\ \mathrm{Prf}_{C_*}(p,1),$$

and hence equality of their consistency assertions.

Proof. The restricted implicational bootstrap is a fixed finite family of proof templates. Its constructor bounds follow from $x, y \le p(x,y) \le 4p(x,y)$. In particular it supplies the usual K, S, Peirce, composition, and exchange templates at arbitrary numerical inputs. The pinned upstream files `M6/Logic/RestrictedImplication.lean` and `PA196/Logic/ImplicationBasis.lean` give these explicit finite constructions. They use no gate. The finite-context deduction construction in `PA196/Logic/Deduction.lean` uses only MP and closed theorems, so it never generalizes an undischarged assumption.

For completeness, explosion also has a uniform gate-free template. From a temporary assumption 1, the K template gives

$$\mathsf{U}(0, \mathsf{N}(\mathsf{U}(0,X))) \to 1 = \exists 0\, \mathsf{U}(0,X).$$

Head h1 then gives X; the finite deduction construction closes this as $1 \to X$. Peirce and explosion supply double-negation elimination, and the three conjunction templates used in Proposition 8.1 follow by ordinary finite deduction. The upstream `PA196/Logic/Connectives.lean` contains those constructors explicitly. The only generalizations in the gate equivalence proof are of already closed conjunction or implication theorems. Thus each direction of that proof expands to a fixed finite template whose parameters are a, b, c.

Scan an input proof from left to right, maintaining the index of the translated conclusion of each old line. A changed gate-axiom line with conclusion $F(a,b,c)$ is replaced by the C_* gate axiom $F_*(a,b,c)$, the common-core template proving $F_* \to F$, and MP. Unchanged axioms are copied. An MP or generalization line is copied with its prior-line references updated. Every translated line group therefore ends in exactly the original numerical conclusion code; there is no formula recoding. The reverse translator uses $F \to F_*$. Extend both algorithms arbitrarily to malformed input records, for example by outputting the empty proof when a required record is missing. List decoding, constructor calculation, finite-template instantiation, appending, and this bounded scan are primitive recursive.

PA verifies each fixed template by its finitely many checking equations and the elementary pairing bounds above. Induction on the input proof length proves the invariant that the remembered

output index has the same numerical conclusion and a valid proof prefix. The only changed axiom has exactly the same numerical side conditions in the two calculi. This proves the two displayed implications inside PA. Existential introduction in each direction proves the final equivalence. This is a syntactic proof-checking argument, and does not use soundness, reflection, or a truth predicate. □

The code-zero identity $\mathsf{I}(0,0) = 0$ does not obstruct this argument: the translation preserves all original conclusion codes, including zero. If the source-level checker exposes a disabled gate as code zero, that result is identical on both sides; it also has the common identity proof $0 \to 0$. The theorem is an ordinary mathematical arithmetization proof. The new translators have not been exported as Lean declarations in this artifact.

The selected head order. The grammar candidate used by the physical compiler additionally emits the six heads in order $[1, 6, 2, 3, 4, 5]$, and writes its sixth head in the exchanged form

$$\mathsf{U}(D, A \to H) \to \big(\mathsf{U}(D, A) \to \mathsf{U}(D, H)\big).$$

The common restricted implicational basis proves exchange independently of h6. Thus the original and exchanged h6 derive one another by one exchange-template application and MP. Add that fixed replacement case to τ and σ to obtain Theorem 8.2 for this selected calculus as well. Permuting the emitted heads does not change the axiom family, but it changes the stack rank used to select each head. The compiler must use the corresponding new ranks. The independent word audit checks this precise permutation and the precise exchanged head; it does not treat an arbitrary permutation as an unchanged numerical command.

Additional equivalent words. Inside Q_*, the three antecedents S_1, S_2, P precede the final falsity code. All six permutations are equivalent by repeated implicational exchange. Within S_1, the two antecedents of $\mathsf{M}(a, b) \to (c \to \mathsf{M}(c, b))$ may also be exchanged. These choices give twelve explicitly certified words with the same macro multiset. They change formula-code layouts and can therefore change grammar sharing even though they prove the same formulas.

8.2 Exact expansion count

The accompanying `distributed_gate.py` reconstructs the original 201-token word from the mathematical heads and compares it with the literal blocks in upstream `PA196/Source/AXProgram.lean`. It checks the pinned source identity and contiguous block boundaries before emitting any variants. The new words each have 199 tokens. Their multiset difference from the original is

$$+1\ \texttt{copyD}\ \ -1\ \texttt{cons}\ \ -2\ \texttt{push}.$$

All increment counts remain the same. The authoritative block lengths are 26 primitive instructions for `copyD`, 27 for `cons`, and 19 for `push`. Consequently

$$26 - 27 - 2 \cdot 19 = -39, \qquad 3812 - 39 = 3773.$$

The enabled gate's contribution drops from 77 to 75 macro tokens. The simplification replaces the negative conjunction's two constant-one constructions and three pairing operations by a second universal constructor and a curried antecedent. The source arithmetic above accounts for their different expansion lengths.

The source is pinned to `jamesyc/BusyBeaver@407cdc356f6b586635e71ac92be394255f4bfea4`. The checked SHA-256 of `PA196/Program.lean` is

`8c7c6c6884ecbb8f56e893404bb81162db15ef3dd6fd3212881a93994d63d872`.

The AX-block source has SHA-256

`8eda8dc0ed0b445c05726990c5a07634464c10ba547fff6bbdba516578ad572c`.

The receipt records all twelve words, exact counts, and the explicit absence of an established physical state count.

What this does not justify. The equality- and membership-vacuity factors have deliberately remained outside the outer universal quantifier. Moving them through that quantifier would require an additional derivation; their reuse of the same bound label cannot be dismissed by an informal freshness claim. The present transformation only distributes a universal over a conjunction and curries antecedents, operations proved above from the unchanged common heads. A source tree, physical allocation, blank startup, universal runtime simulation, and the complete PA consistency calibration remain separate requirements for a smaller TM.

8.3 The 67-rule grammar and the physical allocation problem

The directory `grammar67/` contains an explicit dictionary grammar for the selected 199-token word, together with an independent verifier that does not import an optimizer. It checks the pinned source, reconstructs the original and new formula words, checks that the individual macro blocks are closed, expands every grammar production, and checks the exact final word. The grammar has 26 selected productions of possibly higher arity. Their binary-expansion cost is exactly

$$\sum_{r \text{ selected}} (\text{arity}(r) - 1) = 67.$$

Associating each production as a binary tree therefore gives at most 67 binary rules. This is a constructive upper bound. The time-limited MILP run did not prove optimality, and its floating-point objective is not the certificate: the delivered integer productions are checked separately.

This word has the head order and exchanged h6 specified above. Its exact SHA-256 under the supplied JSON serialization is

`463bd95d864d1473ab282c6034834cbf631b149b16dfd1f4cb3eb5961a1a4dba`.

The separate `axiom_rewrite/verify_selected_word.py` audits that precise formula word and stack order against the mathematical construction.

For comparison, the original AX grammar has 68 internal nodes and six compatible physical folds, giving 62 readers. If a new 67-node allocation retained six compatible folds and all other controls, its count would be

$$134 + 67 - 6 = 195.$$

The premise is essential: changing the word and tree changes the available sharing. The original six folds are not independent credits that can be subtracted from an unrelated grammar.

The first 67-rule dictionary also contains subwords whose cheapest associations force seven consecutive zero address bits. Reusing the existing two-increment reader repairs some static bounds without adding a reader, but static bounds alone remain insufficient. In a tested repaired layout, instruction 1596 ends in six zero address bits and its increment transiently creates another zero; the return scanner then recognizes a false marker. The complete symbolic endpoint check rejects that layout. It is not included as a successful machine. Scanner length, opcode-specific address endings, startup, and compatible helper allocation must be solved together before the grammar gives the requested state-count improvement.

8.4 A complete 198-state implementation of the new gate

The shortest fully checked implementation of the distributed gate produced in this investigation has **198 working states**. This is a concrete compilation result, but it does not improve the existing 196-state bound. The directory `distributed198/` contains the full machine, its exact source tree and compiler, the selected macro word, and a portable verifier.

The dense table `distributed198/candidate.tm` has working states $0, \ldots, 197$, initial state 0, and a separate halt label 198. There are 396 transition rows. Its SHA-256 is

`316c1fd0ade319e3896037f969c18065ecb690339816a5e1defb797ca6751b0c`.

The native table uses initial state 74, interpreter root 84, and halt 254; `labels.json` gives the complete conjugacy. All 198 working states are reachable in the rooted transition graph. No claim that all are visited on the actual blank computation is needed for the count.

Its physical budget is

$$134 \text{ unchanged controls} + 67 \text{ AX readers} - 3 \text{ contextual folds} = 198.$$

The existing native reader 99 supplies the two-increment subtree. The three folds are applied simultaneously and their complete transition edits are recorded in `folds.json`. They retain the original seven-zero scanner. The maximum source address length is 37.

The raw 201-state precursor and the folded 198-state table both pass all 4,908 complete symbolic instruction contracts for the revised 3,773-site program. The folded check uses 1,585,320 symbolic execution steps in total, at most 497 per contract. The arbitrary-gap and whole-register-file interpretation is the same as for the 196-state rotations: all rows used by the inherited register-block contracts are checked unchanged. These counts are symbolic execution counts, not elapsed steps of the blank run.

The literal blank startup takes exactly 58,154 tape transitions. It reaches dense root 75 (native 84), head position -2, source site 25, and the same complete clean frame as PA196, including gap 48 and all 26 spare one-blocks. This equality includes the entire finite tape support and all zeros outside it. The finite startup is used only to establish the simulation's initial condition.

Lemma 8.3 (The selected emitter's nine stack entries)**.** *Let the incoming accumulator be t, the incoming variable registers be d, h, and the three post-prefix parameters be D, H, A. After the selected AX word, the nine newly pushed entries, from top to bottom, are*

$$h5,\ h4,\ h3,\ h2,\ h6',\ h1,\ \rho 8(D, A, H, d),\ U(d, h),\ t,$$

where $h6'$ is the antecedent-exchanged distribution head. The enabled accumulator is the distributed gate $F_*(H, D, A)$; the disabled accumulator is zero under the unchanged guard.

Proof. For a constructor expression X, its mathematical postfix emitter takes an incoming accumulator v and stack w to accumulator X and stack $v :: w$. For a variable or constant-one expression this is the copy/push definition. In the binary-constructor case, the two recursive emissions push the incoming accumulator and then the first child; the constructor consumes the latter, leaving exactly the former. Tag increments do not change the stack. This proves the claim by structural induction on X.

The unchanged 18-token prefix ends with accumulator $\rho 8(D, A, H, d)$ and stack prefix $[U(d, h), t]$. The six successive head emissions have order $[1, 6', 2, 3, 4, 5]$; the final gate emission pushes the last head. Applying the preceding stack identity gives exactly the displayed list. The unchanged guard suffix preserves that stack and either keeps the gate or clears the accumulator. □

An additional independent checker, `axiom_rewrite/audit_physical_binding.py`, imports neither the compiler nor the symbolic runtime. It parses the entire delivered source tree, including its root; proves that it is the pinned PA196 source with only the AX subtree replaced; reconstructs the selected 199-token word; resolves every primitive control edge by subtree intervals; and compares all 3,773 site records with the emitted certificate. It checks every macro occurrence against the pinned macro's relative control graph, the 169 exterior sites and their relocated edges, and the final closing jump. It also independently replays the full blank startup. This audit passed without a discrepancy.

The logical and physical interfaces are now explicit: the machine implements the revised emitter, and the revised numerical calculus has the PA-verifiable proof translations of Theorem 8.2. The ordinary source-level nonhalting transfer uses the inherited scheduler and proof-search argument with the updated head-selection ranks. This package does not export a new Lean build of those arguments or a complete internal PA theorem about the new literal halting sentence. In particular, its relative numerical-calculus theorem does not fill the separately documented baseline consistency-bridge obligations.

8.5 Why the vacuity factors cannot simply be moved under the quantifier

There is a concrete obstruction to replacing the original gate

$$F = \neg(\Delta \to (V \to \forall_c(S \to \neg P)))$$

by

$$F_m = \neg\forall_c(\Delta \to (V \to (S \to \neg P)))$$

using only the retained common heads.

Lemma 8.4 (Failure of common-core factor movement)**.** *Neither implication $F \to F_m$ nor $F_m \to F$ is uniformly derivable from the retained common heads, even under $c \neq a, b$. This does not decide interderivability of the complete gate theories.*

Proof. Use worlds (η, e), where $\eta : \mathbb{N} \to \{0,1\}$ and $e \in \{0,1\}$. Interpret implication classically and define

$$\begin{aligned} E(i,j) &\quad \text{iff} \quad \eta(i) = \eta(j), \\ M(i,j) &\quad \text{iff} \quad e = 0,\ \eta(i) = 0,\ \eta(j) = 1, \\ \forall_i X &\quad \text{iff} \quad X \text{ holds at both } (\eta[i := v], 1 - e), \quad v \in \{0,1\}. \end{aligned}$$

Evaluate natural-number formula codes by their exact PA196 constructors, with code zero true. Since $M(0,0)$ is always false, code one is a genuine falsity in this interpretation. The alias $I(0,0) = 0$ respects classical implication.

Each quantifier accessibility relation is symmetric: a permitted change of coordinate i and color can be reversed by restoring that coordinate and flipping the color again. Hence h1, $\exists_i\forall_i X \to X$, is valid. Two different quantifier relations commute because their coordinate reassignments commute and two color flips cancel; this validates h2. Equality is actual equality at a world, so the two membership-congruence heads h3 and h4 hold. To validate h5, choose the new value of coordinate i to equal the current value of its witness coordinate; when the two indices coincide the equality already holds. Head h6 is ordinary normal-quantifier distribution. The restricted implicational head is a classical tautology. MP and generalization preserve validity. Thus every instance of every retained common head is valid.

Now take numerical parameters $a = 0$, $b = 2$, $c = 1$, and an environment with $\eta(0) = 0$, $\eta(2) = 1$. Both guard inequalities hold. Since the property code $c = 1$ is false, $P = (c \to M(c, b))$ is true at every world. Also Δ is true. The membership-vacuity factor V is false at color zero and true at color one. The separation condition $S = \forall_b((M(a, b) \to P) \wedge (M(c, b) \to c))$ is true at color zero, and at color one it is true exactly when $\eta(c) = 1$. Consequently

	$e = 0$	$e = 1$
F	false	true
F_m	true	false

The two implications therefore both fail in a model of the common core. □

The accompanying `core_countermodel.py` checks the binary-domain, three-label instance exactly. Its 16 worlds suffice for the displayed counterinstance; it checks h1 and h2 for all 65,536 truth sets, all three-label instances of the atomic heads, and all local truth assignments for h6. The preceding argument extends the retained-head validity to arbitrary natural-number labels. A different gate that moves these factors would need a new bootstrap argument establishing the necessary vacuity properties; the successful conjunction- distribution rewrite needs no such step.

9 Using the concise-iPRA work without changing the target

The concise-iPRA material [8] is useful as a source of compilation ideas: explicit syntax, restricted proof interfaces, grammar reuse, and a small trusted checker boundary. Its axiom system cannot simply replace PA's axiom system in this task. A contradiction search for a different theory has that theory's consistency as its natural endpoint; an appropriate uniform calibration to ordinary PA would be an additional theorem.

Two mechanisms in that work are particularly relevant. Its *one-context* and *one-sided transport* presentations recover general equality reasoning from restricted transport rules. Its *direct closure* presentation replaces a witnessed closure rule by a direct universal closure, with an explicit recovery proof. These suggest looking for redundant logical interfaces before optimizing their emitted instruction trees. The weak-cover analysis in the appendix develops this approach for reverse membership rewriting, including the bootstrap that a mere semantic-equivalence argument would omit. The distributed gate in Section 8 gives a source saving while retaining the original PA196 numeric calculus and guard.

These are transfers of proof-design methods. They do not identify the iPRA checker with the PA contradiction search. A shorter high-level checker, a smaller abstract syntax tree, or fewer macro tokens does not automatically produce fewer tape states; the complete binary machine still has to be compiled and verified.

10 Reproduction and evidence boundaries

The mathematical, source, startup, and symbolic-frame checkers use Python 3.10 or later and the standard library. The shared-helper enumeration also requires a C++20 compiler, invoked by its Python wrapper. Run them without `-O`; assertion-based symbolic checks must remain enabled. From the extracted archive directory:

```
python3 verify_all.py
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The archive includes the report source and compiled PDF, full transition tables, reference hashes and notices, exact source programs, verification scripts, and recorded results. The original upstream proof libraries are referenced at their pinned revisions; they are not represented as newly rebuilt in this investigation.

The evidence separates four different levels: literal table and startup checks; complete source control-flow checks; executable all-natural symbolic frame contracts using inherited engine lemmas; and end-to-end formal arithmetic theorems. The first three are supplied for the successful reader rotations and the new 198-state compilation. The fourth is not supplied for those changed tables, and no strict state-count improvement is claimed.

11 Further research questions

1. Can the remaining eighteen six-zero axiom addresses be eliminated while keeping at most 62 physical axiom-body readers and preserving all startup and primitive contracts? This would make the one-state scanner reduction a concrete next candidate.
2. Can source reassociation and retargeting be optimized jointly with helper-row allocation, rather than fixing old reader assignments before searching? Existing compatible-helper restrictions exclude some possible layouts.
3. Can the distributed-gate grammar be allocated with enough compatible folds to beat PA196? The source saving and the 198-state compilation are established here; the lost sharing is now the concrete obstacle. A joint objective must include entry-bit conditions and transient address changes, not only a binary grammar's rule count.
4. Can two or more coordinated helper edits repair a different 195-state quotient with a complete, independently checkable simulation? The two-trace certificates identify why one further row is insufficient for the five fixed first patches; they do not close this larger space.
5. Can the new weak-cover free-instance interface in the next section be compiled with fewer states than the current pointed-separation emitter? The logical recovery should be kept separate from numerical layout scores.
6. Can the reverse PA trace-to-refutation transformation be completed uniformly for the original literal PA196 table, then transported to any verified descendant? This would close the internal biconditional independently of a state-count improvement.

A A logical repair for the weak-cover hybrid

This section supplies a mathematical component for a possible hybrid of U208's weak-cover emitter and PA196's equality-vacuity reduction. It does *not* supply a smaller transition table or an internal PA consistency biconditional. The distinction between ordinary first-order logic and the numeric machine calculus matters: the latter must establish its quantifier and equality rules from its emitted heads.

We use the element-first membership notation $\mathsf{M}(x, s)$ of the U208 role-emitter audit: x belongs to s. A coherent reversal in the numeric encoding of the two arguments is a separate coding change. The primitive symbol E below is not assumed to have reflexivity, symmetry, or transitivity. Define, with a fresh bound label z,

$$\mathrm{Sub}(x, s) := \forall z\, [\mathsf{M}(z, x) \to \mathsf{M}(z, s)].$$

For a property $\phi(x)$, allowing additional parameters, put

$$\mathcal{U}(x,o) := \mathsf{M}(x,o) \to \phi(x), \qquad \mathcal{L}(x;s,o) := \phi(x) \to \big(\mathrm{Sub}(x,s) \to \mathsf{M}(x,o)\big).$$

The weak-cover assertion at source s is

$$\exists o\,[\forall x\,\mathcal{U}(x,o) \;\wedge\; \forall x\,\mathcal{L}(x;s,o)], \tag{3}$$

where o is fresh in the property and source. There are two useful presentations with an additional free point p:

$$W_{\mathcal{L}}(s,p): \quad \exists o\,[\forall x\,\mathcal{U}(x,o) \wedge \forall x\,\mathcal{L}(x;s,o) \wedge \mathcal{L}(p;s,o)], \tag{4}$$
$$W_{\mathcal{U}}(s,p): \quad \exists o\,[\forall x\,\mathcal{U}(x,o) \wedge \forall x\,\mathcal{L}(x;s,o) \wedge \mathcal{U}(p,o)]. \tag{5}$$

All replacements of displayed bound letters avoid capture.

Proposition A.1 (Ordinary logical equivalence)**.** *In ordinary first-order logic, each of* (4) *and* (5) *is equivalent to* (3)*.*

Proof. For a fixed existential witness o, instantiate the corresponding universal conjunct at p and retain that instance as the third conjunct. Conversely, discard the third conjunct. No set-theoretic axiom or equality principle is used. □

In the PA interpretation by binary digits, a witness to (3) is the finite set of all subsets of s satisfying ϕ. Each subset, viewed as a bit code, is at most s. PA's induction constructs the bounded bit-coded comprehension for each fixed formula ϕ. The extra free conjunct is already true of that witness. This is an ordinary mathematical model construction; exporting a uniform arithmetized proof transformation is an additional obligation.

A.1 The retained numeric rules

For the bootstrap we retain PA196's six logical/equality heads other than the choice of orientation of the equality witness, its restricted implicational basis, modus ponens, and closed generalization. The two direct membership rewrites are

$$\mathsf{E}(v,w) \to (\mathsf{M}(v,d) \to \mathsf{M}(w,d)), \tag{6}$$
$$\mathsf{E}(v,w) \to (\mathsf{M}(d,v) \to \mathsf{M}(d,w)). \tag{7}$$

We also retain atomic membership vacuity and add equality vacuity:

$$B \to \forall z B \quad \text{for } B = \mathsf{M}(v,w) \text{ or } \mathsf{E}(v,w), \qquad z \neq v,w.$$

These formulas are obtained by projection from guarded bundles; the numeric guards are the corresponding inequalities of variable labels.

The existing h1 and restricted implicational basis provide explosion and classical implicational reasoning. The h6 head and generalization lift proved implications under a universal quantifier. The h2 head exchanges distinct quantifier dimensions. As in PA196's vacuity proof, these rules extend atomic vacuity to formulas whose free and bound indices exclude the chosen fresh label. They also give the following restricted existential elimination: from $\forall y(A \to C)$ and $\exists y A$, one may infer C when $C \to \forall y C$ is derivable. None of these facts assumes a general substitution rule for primitive equality.

The orientation of the retained witness head is essential. Write

$$\mathsf{W_R}: \quad \exists y\, \mathsf{E}(v,y), \qquad \mathsf{W_L}: \quad \exists y\, \mathsf{E}(y,v).$$

Here $\mathsf{W_R}$ is PA196's mirrored witness; $\mathsf{W_L}$ is the original orientation. The free-lower construction below uses $\mathsf{W_R}$, and the free-upper construction uses $\mathsf{W_L}$. The other pairing of a free instance with a witness orientation is not certified by the argument in this section.

A.2 Two restricted specialization lemmas

Let $\chi(t)$ be one of $\mathsf{M}(d,t)$, $\mathsf{M}(t,d)$, or $\mathsf{M}(t,t)$. The two direct rewrites give

$$\mathsf{E}(v,w) \to (\chi(v) \to \chi(w)).$$

For the diagonal atom, apply the two coordinate rewrites in succession. All freshness conditions stated below concern numeric variable labels.

Lemma A.2 (Negative upper specialization)**.** *With $\mathsf{W_R}$, if $y \neq v, o$ and $d \neq y$ when d occurs, then*

$$\forall y\, [\mathsf{M}(y,o) \to \neg\chi(y)] \;\to\; [\mathsf{M}(v,o) \to \neg\chi(v)]$$

is derivable.

Proof. Direct rewriting gives, under $\mathsf{E}(v,y)$, both $\mathsf{M}(v,o) \to \mathsf{M}(y,o)$ and $\chi(v) \to \chi(y)$. Classical reasoning therefore gives

$$\mathsf{E}(v,y) \to ((\mathsf{M}(y,o) \to \neg\chi(y)) \to (\mathsf{M}(v,o) \to \neg\chi(v))).$$

Generalize at y and use h6 with the displayed universal premise. The conclusion $\mathsf{M}(v,o) \to \neg\chi(v)$ is vacuous at y. Eliminate the witness $\exists y\, \mathsf{E}(v,y)$ by the restricted existential rule. This is a proof of this particular specialization, not an invocation of unrestricted universal instantiation. □

Lemma A.3 (Negative lower specialization)**.** *With $\mathsf{W_L}$, let $\phi(t) = \neg\chi(t)$. If y differs from v, s, o and from d when present, and the subset binder o is fresh for y, v, s, then*

$$\forall y\, \mathcal{L}(y; s, o) \;\to\; \mathcal{L}(v; s, o)$$

is derivable. The same letter o may name the free output in the membership atom and the locally bound variable inside Sub*.*

Proof. Under $\mathsf{E}(y,v)$, the direct rewrites imply

$$\neg\chi(v) \to \neg\chi(y), \qquad \mathsf{M}(y,o) \to \mathsf{M}(v,o).$$

They also imply $\mathrm{Sub}(v,s) \to \mathrm{Sub}(y,s)$ under that equality assumption. Indeed, $\mathsf{E}(y,v) \to (\mathsf{M}(o,y) \to \mathsf{M}(o,v))$ is a direct rewrite. Generalization at o, equality vacuity at o, and h6 lift it to $\mathsf{E}(y,v) \to \mathrm{Sub}(y,v)$. Combine it pointwise with $\mathrm{Sub}(v,s)$, using h6, to obtain $\mathrm{Sub}(y,s)$. Consequently

$$\mathsf{E}(y,v) \to (\mathcal{L}(y; s, o) \to \mathcal{L}(v; s, o))$$

is derivable. Generalize at y, combine with $\forall y \mathcal{L}(y; s, o)$, and eliminate $\exists y \mathsf{E}(y,v)$ using $\mathsf{W_L}$ and vacuity of $\mathcal{L}(v; s, o)$ at y. Only the explicitly displayed rewrites and quantifier rules have been used. □

A.3 Recovery of reverse membership rewriting

Proposition A.4 (Matching free instances repair the bootstrap)**.** *Either of the following additions to the retained rules derives both reverse membership rewrites:*

1. $W_{\mathcal{L}}$ *together with* $\mathsf{W_R}$*;*
2. $W_{\mathcal{U}}$ *together with* $\mathsf{W_L}$*.*

More precisely, for every a, b, d *they derive*

$$\mathsf{E}(a,b) \to (\mathsf{M}(d,b) \to \mathsf{M}(d,a)), \qquad \mathsf{E}(a,b) \to (\mathsf{M}(b,d) \to \mathsf{M}(a,d)).$$

The old primitive equality-transitivity head is not used.

Proof. For $a = b$ the conclusions are propositional tautologies. Suppose $a \neq b$.

First use $W_{\mathcal{L}}$ and $\mathsf{W_R}$. For $d \neq a$, take $\phi(t) = \neg\chi(t)$, source $s = b$, and distinguished point $p = a$. Choose the output label o fresh for a, b, d. By direct rewriting, equality vacuity, generalization and h6,

$$\mathsf{E}(a,b) \to \mathrm{Sub}(a,b).$$

Assume $\mathsf{E}(a,b)$, $\chi(b)$ and $\neg\chi(a)$. The free lower conjunct gives $\mathsf{M}(a,o)$, the direct first-coordinate rewrite gives $\mathsf{M}(b,o)$, and Lemma A.2, applied to the upper universal conjunct, gives $\neg\chi(b)$. This is a contradiction. The final implication has no index o, so its vacuity permits elimination of the existential output. Thus $\mathsf{E}(a,b) \to (\chi(b) \to \chi(a))$ is obtained for $d \neq a$ and for the diagonal atom. If $d = a$, each of $\mathsf{M}(a,b)$ and $\mathsf{M}(b,a)$ transfers directly to $\mathsf{M}(b,b)$ under $\mathsf{E}(a,b)$. The diagonal reverse implication gives $\mathsf{M}(a,a)$, resolving both omitted cases.

Next use $W_{\mathcal{U}}$ and $\mathsf{W_L}$. For $d \neq b$, take $\phi(t) = \neg\chi(t)$, source $s = a$, and distinguished point $p = b$, again with a fresh output o. Under $\mathsf{E}(a,b)$, $\chi(b)$ and $\neg\chi(a)$, Lemma A.3 specializes the lower universal conjunct to a. The formula $\mathrm{Sub}(a,a)$ follows by generalizing an implicational identity. Hence $\mathsf{M}(a,o)$ follows. Direct rewriting gives $\mathsf{M}(b,o)$, and the free upper conjunct at b gives $\neg\chi(b)$, a contradiction. Eliminate the existential output as before. For $d = b$, the diagonal reverse implication takes $\mathsf{M}(b,b)$ to $\mathsf{M}(a,a)$; the direct rewrites then yield $\mathsf{M}(b,a)$ and $\mathsf{M}(a,b)$ respectively, settling the two omitted cases. □

This provides the two directions of membership rewriting needed for PA196's defined-neighborhood equality. Generalization and equality vacuity then give the implication from primitive E to that defined relation. The relevant equality witness is transported to the defined relation by existential monotonicity, reversing the *defined* relation when necessary. No claim that primitive E itself has a separate symmetry or transitivity theorem is required.

A.4 Native codes and the remaining obligations

The argument respects the special coupling between a property code and an output-variable label. In the U208 role-emitter notation, the same number a is both the property code and the output label; b is the source label and c the tested label. Put

$$\begin{aligned} e &= \mathsf{M}(c,a), & G &= \forall a\,[\mathsf{M}(a,c) \to \mathsf{M}(a,b)],\\ S &= e \to a, & U &= \forall c\, S,\\ R &= a \to (G \to e), & L &= \forall c\, R. \end{aligned}$$

Here a in formula position denotes the decoded formula with code a. The two revised cover tails are

$$H_R = \forall a\,[U \to (L \to (R \to \bot))], \qquad H_S = \forall a\,[U \to (L \to (S \to \bot))].$$

With $\Delta = \mathsf{E}(a,b) \to \forall c\mathsf{E}(a,b)$ and $V = \mathsf{M}(a,b) \to \forall c\mathsf{M}(a,b)$, emit respectively

$$\neg(\Delta \to (V \to H_R)), \qquad \neg(\Delta \to (V \to H_S))$$

under the original guard $c \neq a, b$. Keep truth as the disabled output. The free-lower version uses the mirrored equality witness, and the free-upper version uses the original witness.

For the negative-property instances used in the proof, the property code exceeds its displayed atom indices. If it equals the other distinguished label, use the equivalent triple negation instead of a single negation; its code is strictly larger. The contraposition argument still supplies the same negative-property specialization, and the output label is now fresh for all indices in the final conclusion. The source and test labels are different because the case $a = b$ in the proof was already discharged propositionally. Thus the native guard does not remove any case needed for Proposition A.4.

In ordinary first-order logic the new tails add only an instance of an existing universal conjunct. This observation also covers the original role-emitter's exceptional equality of its source and output labels: the native free instance has the same local binding as the body it copies. The separate calibration of that exceptional old weak-cover instance must still be retained.

Scope. The new component is a noncircular derivation of the reverse-rewriting interface for two precisely specified emitter variants. A complete construction must still supply the exact AX stack contract and selection ranks; complete numeric formula/proof translations, including any argument or tag recoding; a physical allocation with correct startup and universal runtime contracts; and both coded trace/proof directions in PA. This section supplies none of those by implication. In particular, it establishes neither a machine with fewer than 196 states nor the full PA-provable nonhalting–consistency biconditional.

References

[1] James Chang, *Nonhalting and PA-independence for a 196-state Turing machine*, revision 407cdc356f6b586635e71ac92be394255f4bfea4. https://github.com/jamesyc/BusyBeaver/blob/407cdc356f6b586635e71ac92be394255f4bfea4/paper/pa196.tex.

[2] James Chang, PA196 theorem statements and reproduction instructions, same revision. https://github.com/jamesyc/BusyBeaver/blob/407cdc356f6b586635e71ac92be394255f4bfea4/docs/result.md.

[3] James Chang, PA209 predecessor, revision 69d98241a721ead20b4f0b17df9062950b40fbfe. https://github.com/jamesyc/BusyBeaver/tree/69d98241a721ead20b4f0b17df9062950b40fbfe.

[4] Vladimir Reshetnikov, ProveIt incoming research packages, pinned revision f1f076e31995f10460838d1846456f4bb9c8ae80. https://github.com/VladimirReshetnikov/ProveIt/tree/f1f076e31995f10460838d1846456f4bb9c8ae80/docs/incoming. Exact archive/member identities appear in the included input manifest.

[5] ProveIt, PA196 consistency bridge audit, same pinned revision. https://github.com/VladimirReshetnikov/ProveIt/blob/f1f076e31995f10460838d1846456f4bb9c8ae80/Computability/BusyBeaver/Research/PA/pa196-vs-u208/PA-BRIDGE.md.

[6] ProveIt, earlier PA196/U208 comparison and recorded search boundaries, same pinned revision. https://github.com/VladimirReshetnikov/ProveIt/tree/f1f076e31995f10460838d1846456f4bb9c8ae80/Computability/BusyBeaver/Research/PA/pa196-vs-u208.

[7] James Chang, generic tape runtime and native register contracts, PA196 revision cited above. https://github.com/jamesyc/BusyBeaver/blob/407cdc356f6b586635e71ac92be394255f4bfea4/M6/Runtime.lean.

[8] ProveIt, *ConciseIPRA*, same ProveIt revision. https://github.com/VladimirReshetnikov/ProveIt/tree/f1f076e31995f10460838d1846456f4bb9c8ae80/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.